average, weighted toward the newest observation. This way, if things change over time, we would be able to track them to some degree. We won't implement this slightly more complex variant here, but we will encounter nonstationary problems later in the book.

2.2.3. Softmax selection policy

Imagine another type of bandit problem: A newly minted doctor specializes in treating patients with heart attacks. She has 10 treatment options, of which she can choose only 1 to treat each patient she sees. For some reason, all she knows is that these 10 treatments have different efficacies and risk profiles for treating heart attacks—she doesn't know which one is the best yet. We could use the *n*-armed bandit algorithm from the previous solution, but we might want to reconsider our ε-greedy policy of randomly choosing a treatment once in a while. In this new problem, randomly choosing a treatment could result in patient death, not just losing some money. We really want to make sure we don't choose the worst treatment, but we still want some ability to explore our options to find the best one.

This is where a *softmax* selection might be most appropriate. Instead of just choosing an action at random during exploration, softmax gives us a probability distribution across our options. The option with the largest probability would be equivalent to the best arm action in the previous solution, but it will also give us some idea about which are the second and third best actions, for example. This way we can randomly choose to explore other options while avoiding the very worst options, since they will be assigned tiny probabilities or even 0. Here's the softmax equation:

Table 2.4. The softmax equation

Math	Pseudocode

$\Pr(A) = \frac{e^{Q_k(A)/\tau}}{\sum_{i=1}^{n} e^{Q_k(i)/\tau}}$	`def softmax(vals, tau):`
	`  softm = pow(e, vals / tau) / sum(pow(e, vals / tau))`
	`  return softm`